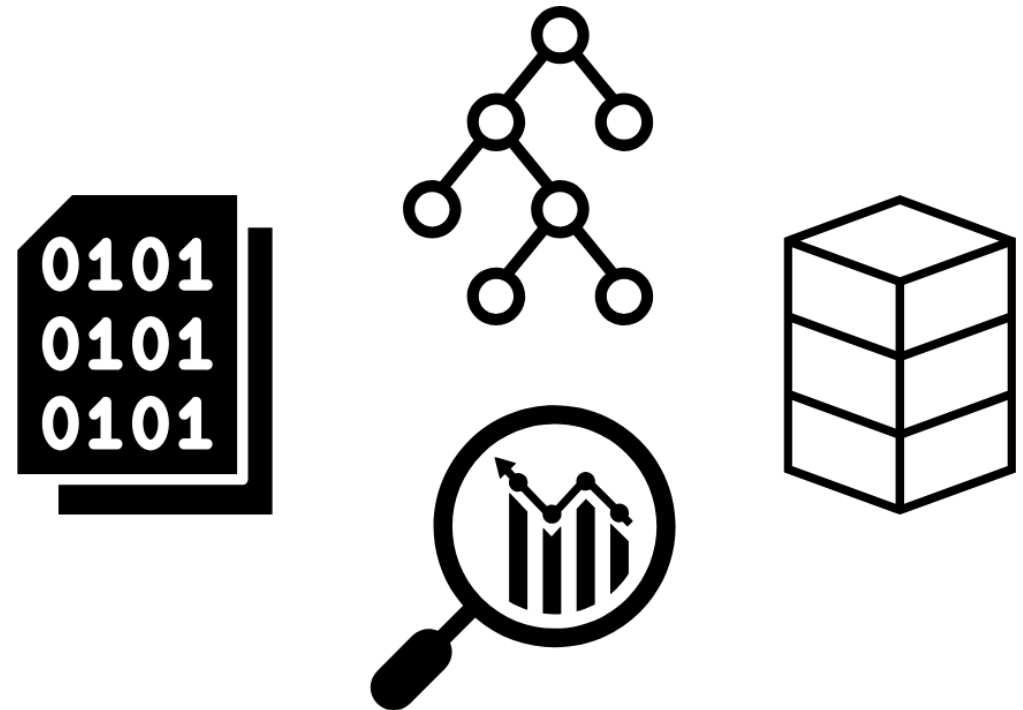


Estrutura de Dados

Introdução à Linguagem C



Por que C?

Estrutura de Dados não é uma disciplina para aprender uma linguagem de programação.

A linguagem será uma ferramenta para compreender **dados, memória, algoritmos e estruturas.**

O que a escolha de C torna mais visível?

O que é a linguagem C?

- Linguagem de programação de **propósito geral**
- Caracterizada por **expressividade, portabilidade e eficiência**
- Muito utilizada em software de baixo nível e sistemas
- Permite maior controle sobre recursos da máquina

C combina recursos de alto nível com uma relação próxima com a representação e manipulação dos dados no computador.

De onde veio o C?



Ano	1972
Local	Bell Labs
Principal nome	Dennis Ritchie
Contexto inicial	UNIX / DEC PDP-11

C e Python

	Python	C
Tipos	Tipagem dinâmica	Tipos declarados (tipagem estática)
Execução	Runtime / interpretador	Compilado
Memória	Maior abstração	Maior controle
Aprendizado	Foco na lógica	Lógica + memória + fundamentos

A lógica de programação continua sendo aproveitada. O que muda é o quanto do processo de implementação é explicitado para o programador.

Código-fonte

```
#include <stdio.h>

int main(void) {
    printf("Olá, mundo!\n");

    return 0;
}
```

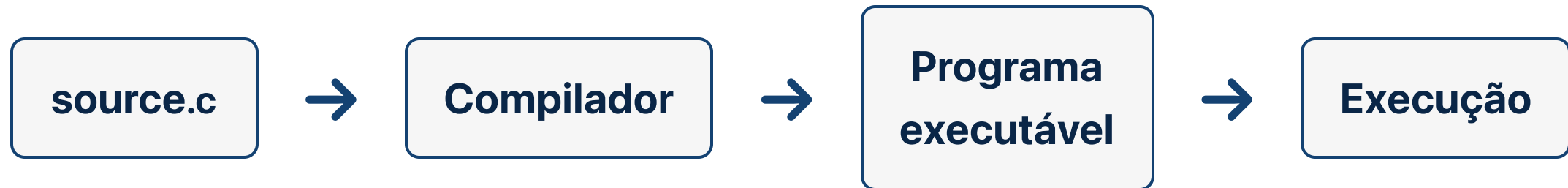
Arquivo: hello.c

Terminal Problemas Postman Con

```
● → exemplos gcc hello.c -o hello
● → exemplos ./hello
    Olá, mundo!
○ → exemplos
```

O código-fonte é o texto que escrevemos. Ainda não é o programa executável.

Como o código vira um programa?

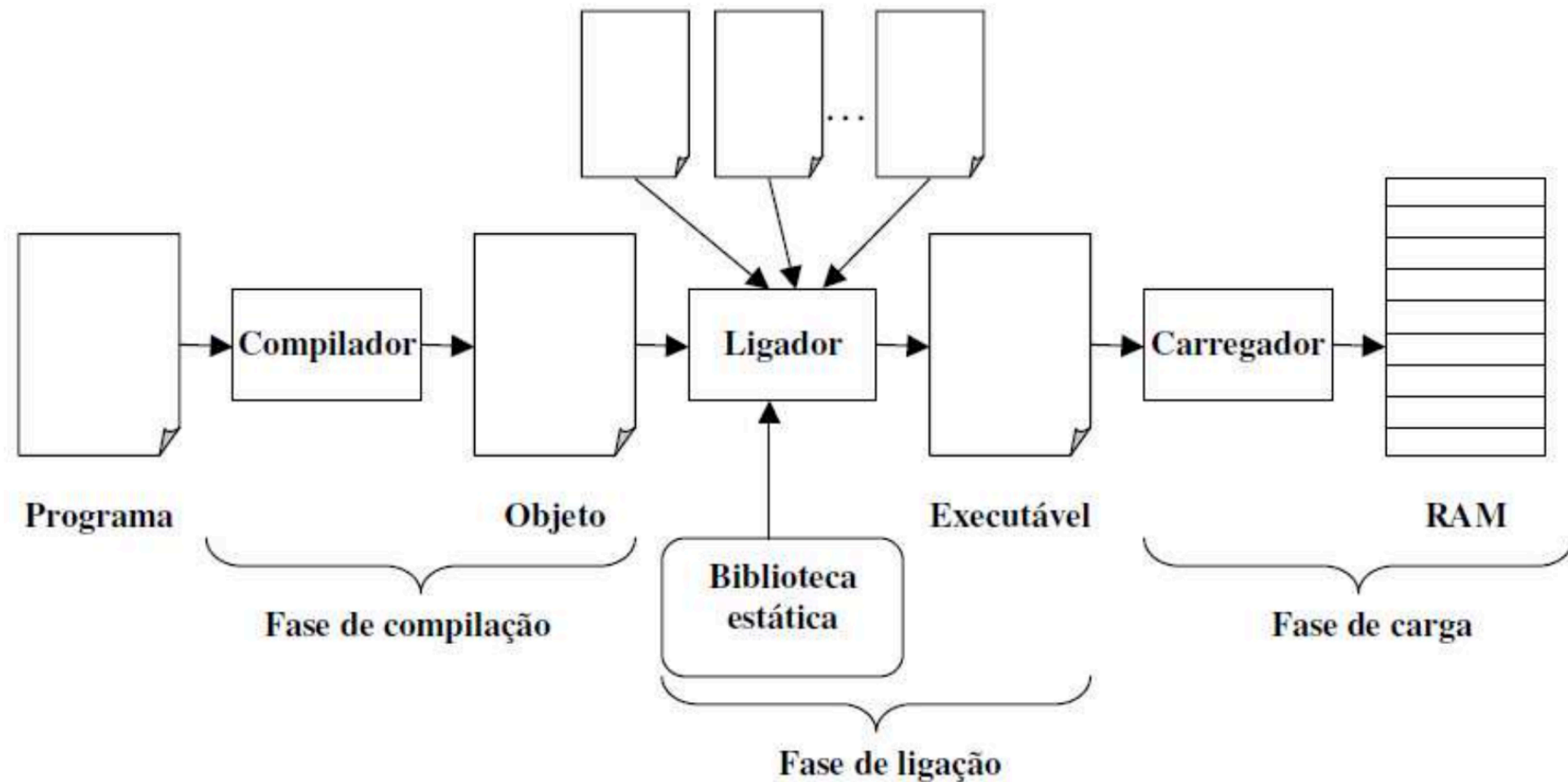


Compilar é transformar o código-fonte em código executável.

O Processo de Compilação

1. Recebe o código-fonte
2. Analisa a estrutura e verifica erros
3. Traduz para a arquitetura-alvo
4. Participa da geração do programa executável

O Processo de Compilação



E quando existe um erro?

```
#include <stdio.h>

int main(void) {
    printf("Olá, mundo!\n")

    return 0;
}
```

O compilador encontra um erro de sintaxe antes que o programa seja executado.

Erro de compilação $\neq$ erro de execução $\neq$ erro lógico

Estrutura básica de um programa C

```
#include <stdio.h>

int main(void) {
    printf("Olá, mundo!\n");

    return 0;
}
```

- `#include <stdio.h>` → biblioteca
- `main` → função principal

- `{ ... }` → bloco de instruções
- `return 0` → encerramento da função

Entendendo o `main`

```
int main(void)
```

int

tipo de retorno

main

nome da função

void

sem argumentos

Programas manipulam dados



Constantes

```
const int ANO = 2023;  
const float PI = 3.14159f;  
const char INICIAL = 'L';
```

Variáveis

```
int idade = 20;
```

Tipos de dados primitivos

Tipo	Representa	Tamanho*	Exemplo
int	Números inteiros	4 bytes	20
float	Números reais	4 bytes	3.14f
double	Números reais com maior precisão	8 bytes	3.141592
char	Um caractere	1 byte	'L'

* Tamanho pode variar conforme a implementação e a arquitetura.

Tipos de dados primitivos

- Em C, na verdade, podemos resumir os tipos de dados primitivos a dois tipos: **inteiros** e **reais**.
- O que acontece com os outros tipos é que eles são **representações específicas** de inteiros e reais.
- O tipo `char` é uma representação de inteiro, pois cada caractere é representado por um código numérico, conforme a famosa **tabela ASCII**.
 - Por exemplo, o caractere `'A'` é representado pelo número `65`. Você pode verificar isso com o código:

```
char letra = 'A';  
printf("%d\n", letra); // Saída: 65
```

- O tipo `double` é uma representação de real com maior precisão, pois ocupa mais memória (o dobro de `float`).

Tipos de dados primitivos

- Execute o código abaixo para descobrir o tamanho de cada tipo de dado primitivo em seu ambiente de desenvolvimento:

```
printf("char:    %zu bytes\n", sizeof(char));  
printf("int:     %zu bytes\n", sizeof(int));  
printf("float:   %zu bytes\n", sizeof(float));  
printf("double: %zu bytes\n", sizeof(double));
```

sizeof

Retorna o tamanho, em **bytes**, de um tipo ou expressão.

O tamanho de alguns tipos depende da implementação e da arquitetura.

Um pouco sobre vetores

- **Vetores** são estruturas de dados que armazenam **uma coleção de elementos do mesmo tipo**.
- Os elementos são armazenados em **endereços de memória contíguos**, ou seja, um ao lado do outro.
- Em C, podemos declarar um vetor da seguinte forma:

```
int numeros[5]; // declara um vetor de 5 inteiros
```

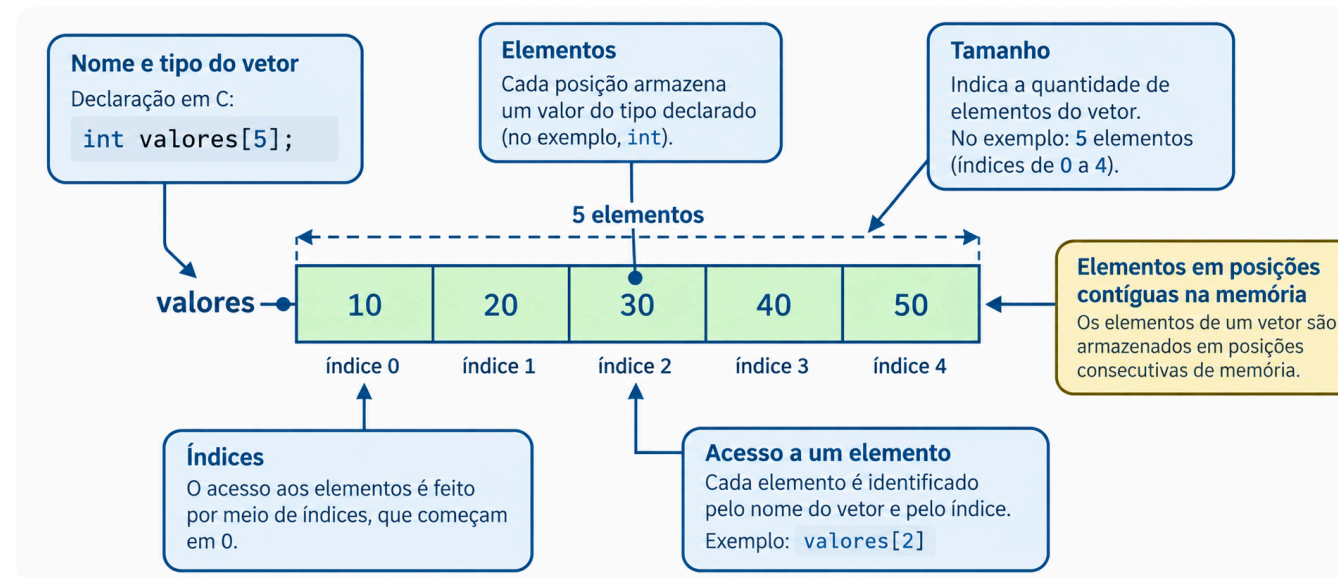
- Podemos inicializar o vetor no momento da declaração:

```
int numeros[] = {20, 13, 42, 5, 18};
```

- nesse caso, o tamanho do vetor é definido pelo número de elementos fornecidos entre chaves.

Um pouco sobre vetores

- Vetor é uma **estrutura estática**, ou seja, seu tamanho é definido no momento da declaração e/ou inicialização e não pode ser alterado depois.
- O acesso aos elementos do vetor é feito por meio de **índices**, que começam em 0.



Um pouco sobre vetores

- Exemplo de acesso a elementos de um vetor:

```
int numeros[] = {20, 13, 42, 5, 18};

printf("%d\n", numeros[0]); // Saída: 20
printf("%d\n", numeros[1]); // Saída: 13
printf("%d\n", numeros[2]); // Saída: 42
printf("%d\n", numeros[3]); // Saída: 5
printf("%d\n", numeros[4]); // Saída: 18
```

- Alterando o valor de um elemento do vetor:

```
numeros[2] = 99; // altera o valor do terceiro elemento para 99
printf("%d\n", numeros[2]); // Saída: 99
```

 Veremos mais sobre vetores em aulas futuras.

Contantes

- Em C, podemos declarar **constantes** utilizando a palavra-chave `const`.
- Uma constante é, em resumo, uma variável somente de leitura, ou seja, seu valor não pode ser alterado após a inicialização.
- Exemplo de declaração de constantes:

```
const int ANO = 2026;  
const float PI = 3.14159f;
```

- se, mais adiante neste código, você tentar, por exemplo, `ANO = 2024;`, o compilador irá gerar um erro, pois `ANO` é uma constante.
- mesmo não sendo uma regra, é uma boa prática declarar constantes com **letras maiúsculas**, para diferenciá-las de variáveis comuns.

Cuidado com o lixo de memória

- Em C, quando declaramos uma variável sem inicializá-la, ela pode conter um **valor indefinido**, ou seja, um valor aleatório que estava na memória naquele endereço.
- Costumamos chamar esse valor de **lixo de memória**.
- Isso ocorre porque a memória que a variável ocupa, pode ter sido utilizada anteriormente por outro programa ou outra parte do código, e o valor antigo ainda está lá. Observe o exemplo abaixo:

```
int a, b, c; // declara as variáveis, mas não as inicializa
a = 2; // inicializa a variável 'a'
c = a + b; // 'b' não foi inicializada, então seu valor é indefinido
```

- como a variável **b** não foi inicializada, o valor de **c** será indefinido, pois depende do valor aleatório que estava na memória naquele endereço.
- isso pode ocasionar um comportamento inesperado do programa, e é por isso que é importante **sempre inicializar as variáveis** antes de ler seus valores.

Entrada e saída de dados

- A entrada e saída de dados é feita por meio das funções `printf` e `scanf`, que fazem parte da biblioteca padrão `<stdio.h>`.
- Para usar essas funções, precisamos incluir a biblioteca no início do programa:

```
#include <stdio.h>
```

- mais adiante, vamos estudar sobre bibliotecas. De início, basta saber que `<stdio.h>` é uma biblioteca que contém funções para **entrada e saída de dados**.
- a função `printf` é utilizada para **exibir dados na tela**.
- a função `scanf` é utilizada para **ler dados digitados pelo usuário**.
- sem a biblioteca `<stdio.h>`, essas funções não estariam disponíveis e o código não compilaria.

Saída de dados e especificadores de formato

- Os **especificadores de formato** são utilizados nas funções `printf` e `scanf` para indicar o tipo de dado que será exibido ou lido.
- Exemplo de saída de dados com `printf`:

```
int idade = 20;  
float altura = 1.75f;  
printf("Idade: %d\n", idade);  
printf("Altura: %f\n", altura);
```

- perceba que usamos `%d` para indicar que o valor de `idade` é um **inteiro**, e `%f` para indicar que o valor de `altura` é um **float**.
- poderíamos ainda imprimir esses dados em uma única chamada de `printf`:

```
printf("Idade: %d, Altura: %f\n", idade, altura);
```

Saída de dados e especificadores de formato

```
#include <stdio.h>

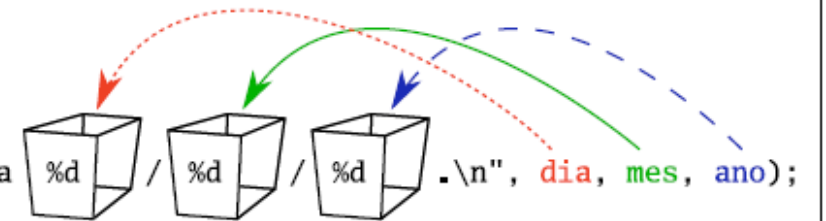
int main(void) {
    int dia, mes, ano;

    dia = 15;
    mes = 8;
    ano = 2026;

    printf("Hoje é dia %d/%d/%d.\n", dia, mes, ano);

    return 0;
}
```

```
printf("Hoje é dia %d / %d / %d .\n", dia, mes, ano);
```



Saída de dados e especificadores de formato

- Abaixo estão os principais especificadores de formato utilizados em C:

Especificador	Uso
%d	inteiro
%f	real
%c	caractere
%s	string
%lf	real de dupla precisão (double)
%g	real em formato automático (notação científica ou decimal)

Saída de dados e especificadores de formato

- Outro detalhe importante é que podemos controlar a **quantidade de casas decimais** exibidas para números reais.
- Por exemplo, para exibir apenas duas casas decimais, podemos usar o especificador `%.2f` :

```
float altura = 1.75f;  
printf("Altura: %.2f\n", altura); // Saída: Altura: 1.75
```

- Também podemos controlar a **largura mínima** de exibição de um valor.
- Por exemplo, para exibir um número com pelo menos 5 caracteres de largura, podemos usar o especificador `%5d` :

```
int idade = 20;  
printf("Idade: %5d\n", idade); // Saída: Idade:    20 (3 espaços à esquerda)
```

Saída de dados e sequências de escape

- As **sequências de escape** são utilizadas para representar caracteres especiais em strings.
- Você deve ter notado que usamos `\n` muitas vezes ao final de strings na função `printf`.
- O `\n` representa uma **nova linha**, ou seja, ele faz com que o cursor vá para a próxima linha no terminal. O código abaixo implicaria na exibição das duas saídas em uma única linha:

```
printf("Idade: %d", idade);  
printf("Altura: %.2f", altura);  
// Saída: Idade: 20Altura: 1.75
```

- para evitar isso, usamos `\n` para quebrar a linha:

```
printf("Idade: %d\n", idade);  
printf("Altura: %.2f\n", altura);  
// Saída:  
// Idade: 20  
// Altura: 1.75
```

Saída de dados e sequências de escape

- Outras sequências de escape comuns são:

Sequência	Significado
<code>\n</code>	newline (nova linha)
<code>\t</code>	tabulação (tab)
<code>\\</code>	barra invertida (backslash)
<code>\"</code>	aspas duplas (double quotes)
<code>\'</code>	aspas simples (single quotes)

Entrada de dados com `scanf`

```
int idade;  
  
printf("Digite sua idade: ");  
scanf("%d", &idade);  
  
printf("Você tem %d anos.\n", idade);
```

%d

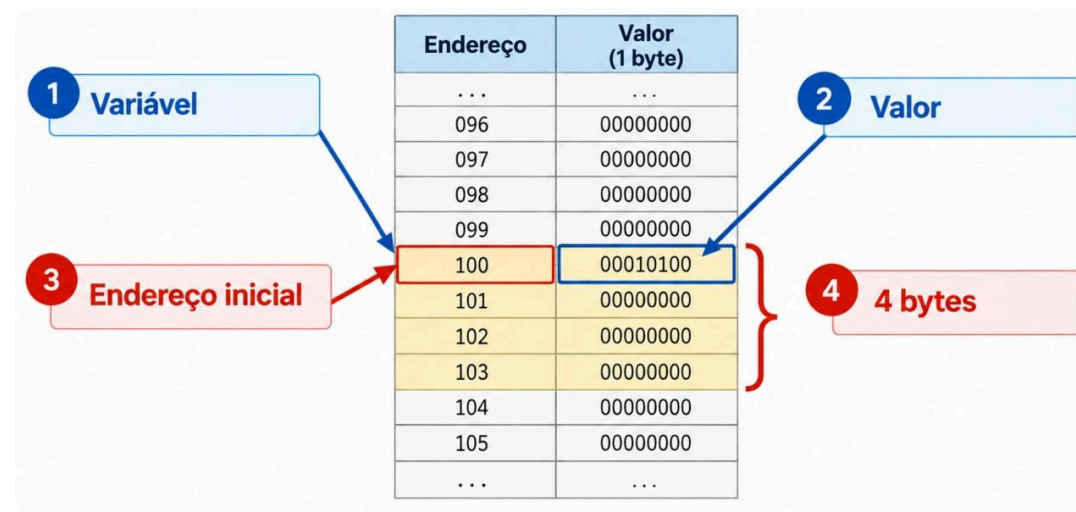
Indica o tipo de dado esperado.

&idade

Indica onde o valor deverá ser armazenado.

E esse &?

```
int idade = 20;  
printf("%p\n", &idade);
```



& é um **operador de endereço**. Logo, **&idade** representa o endereço da variável **idade**.

O especificador de formato para endereços é **%p**. Mais adiante vamos estudar **ponteiros** em profundidade.

Operadores aritméticos

- Os operadores aritméticos são utilizados para realizar operações matemáticas básicas em C.
- Basicamente, são os mesmos operadores que você já conhece de outras linguagens de programação, como Python.

Operador	Significado
+	adição
-	subtração
*	multiplicação
/	divisão
%	módulo (resto da divisão)

Operadores aritméticos - Incremento e Decremento

- Diferente do Python, em C temos operadores de **incremento** e **decremento**, que são utilizados para aumentar ou diminuir o valor de uma variável em 1, o que pode ser útil em loops e outras situações.
- **Incremento:** `++` (exemplo: `idade++` ou `++idade`)
- **Decremento:** `--` (exemplo: `idade--` ou `--idade`)
- **Exemplos:**

```
int idade = 20;  
idade++; // idade agora é 21  
idade--; // idade agora é 20
```

- **Pré-incremento** (`++idade`) e **pós-incremento** (`idade++`) têm diferenças sutis. Por exemplo, o código `printf("%d\n", idade++);` imprime o valor atual de `idade` e depois incrementa, enquanto `printf("%d\n", ++idade);` incrementa primeiro e depois imprime o novo valor.

Operadores aritméticos - Abreviações de atribuição

- Atribuições de valor que envolvem o valor original da variável a ser atualizada, podem ser abreviadas com operadores de atribuição combinados.
- Imagine que você queira somar 5 ao valor de uma variável `x`. Você poderia fazer isso assim:

```
x = x + 5;
```

- Mas, para evitar repetir o nome da variável, podemos usar o operador de atribuição combinado `+=`:

```
x += 5; // equivalente a x = x + 5
```

Operadores aritméticos - Abreviações de atribuição

- Quando $\odot$ é um operador aritmético, podemos abreviar $a = a \odot b$ para $a \odot= b$.
- Dessa forma, temos:

Abreviação	Expressão completa
$a += b$	$a = a + b$
$a -= b$	$a = a - b$
$a *= b$	$a = a * b$
$a /= b$	$a = a / b$
$a \% = b$	$a = a \% b$

Bloco de código e escopo

- Um **bloco de código** é um trecho de código delimitado por chaves `{}`.
- Ele define o **escopo** das variáveis declaradas dentro dele.
- **Exemplo:**

```
int somar(int a, int b) {  
    int resultado = a + b; // 'resultado' só existe dentro deste bloco  
    return resultado;  
}
```

- a variável `resultado` só pode ser acessada dentro da função `somar`. Fora dela, ela não existe. Ou seja, `resultado` tem **escopo local** à função `somar`.

Bloco de código e escopo

Escopo local x escopo global

- **Escopo local:** variáveis declaradas dentro de um bloco de código só podem ser acessadas dentro desse bloco, inclusive em blocos aninhados.
- **Escopo global:** variáveis declaradas fora de qualquer bloco de código podem ser acessadas em qualquer lugar do programa.

Controle de fluxo - Desvios Condicionais

if / else

- A sintaxe básica da estrutura `if / else`, em C, é:

```
if (condição) {  
    // código a ser executado se a condição for verdadeira  
} else {  
    // código a ser executado se a condição for falsa  
}
```

- perceba o uso das chaves `{}` para delimitar o bloco de código que será executado em cada caso.

Controle de fluxo - Desvios Condicionais

Múltiplas condições: `else if`

- Podemos testar múltiplas condições utilizando `else if`:

```
int idade;
printf("Digite sua idade: ");
scanf("%d", &idade);

if (idade < 18) {
    // executado quando idade for menor que 18
} else if (idade < 65) {
    // executado quando idade for maior ou igual a 18 e menor que 65
} else {
    // executado quando idade for maior ou igual a 65
}
```

Controle de fluxo - Desvios Condicionais

Múltiplas condições: `switch / case`

- A estrutura `switch / case` é uma alternativa ao `if / else if` quando temos múltiplas condições baseadas em um mesmo valor.

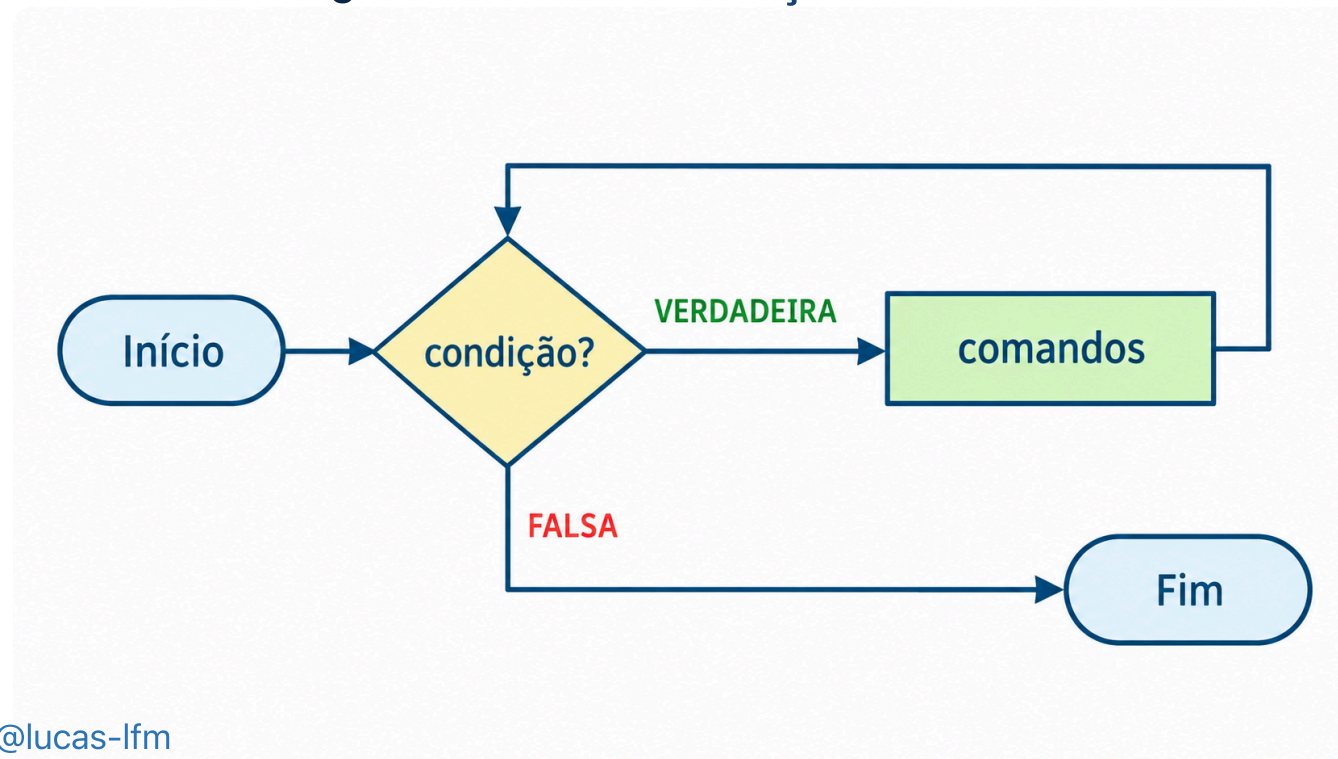
- Exemplo:**

```
int dia = 3;
switch (dia) {
    case 1:
        printf("Domingo\n");
        break;
    case 2:
        printf("Segunda-feira\n");
        break;
    case 3:
        printf("Terça-feira\n");
        break;
    default:
        printf("Outro dia\n");
}
```

- O `switch` recebe uma expressão e executa o bloco correspondente ao valor dessa expressão.
- Cada `case` representa um valor possível da expressão.
- O `break` é utilizado para sair do `switch` após executar o bloco correspondente.
- O `default` é opcional e é executado caso nenhum dos `case` seja atendido.

Controle de fluxo - Laços de repetição

- **Relembrando:** Os laços de repetição são estruturas que **permitem executar um bloco de código várias vezes**, enquanto uma condição for verdadeira.
- Cada execução do bloco de código é chamada de **iteração**.



Controle de fluxo - Laços de repetição

while

- O laço `while` executa um bloco de código enquanto uma condição for verdadeira.
- A sintaxe básica é:

```
while (condição) {  
    // código a ser executado enquanto a condição for verdadeira  
}
```

- Exemplo de `while` com contador:

```
int numeros[5] = {20, 13, 42, 5, 18};  
int contador = 0;  
while (contador < 5) {  
    printf("Número: %d\n", numeros[contador]); // exibe o número atual  
    contador++; // incrementa o contador  
}
```

Controle de fluxo - Laços de repetição

while

- Também podemos usar o `while` para ler dados do usuário até que uma condição seja atendida. Por exemplo, podemos ler números inteiros até que o usuário digite um número negativo:

```
int numero;
printf("Digite um número (negativo para sair): ");
scanf("%d", &numero);

while (numero >= 0) {
    printf("Número lido: %d\n", numero);
    printf("Digite outro número (negativo para sair): ");
    scanf("%d", &numero);
}

printf("Programa encerrado.\n");
```

Controle de fluxo - Laços de repetição

`while` com valor sentinela

- Perceba que no caso anterior, não sabemos quantas vezes o laço será executado.
- Nessas situações, o `while` é a melhor escolha, pois ele executa o bloco de código enquanto a condição for verdadeira.
- Diferente do uso de um contador, aqui contamos com um **valor sentinela**.
 - chamamos de **valor sentinela** aquele valor que indica o fim da entrada de dados.
 - esse valor pode ser fornecido pelo usuário ou resultado de uma operação.

Controle de fluxo - Laços de repetição

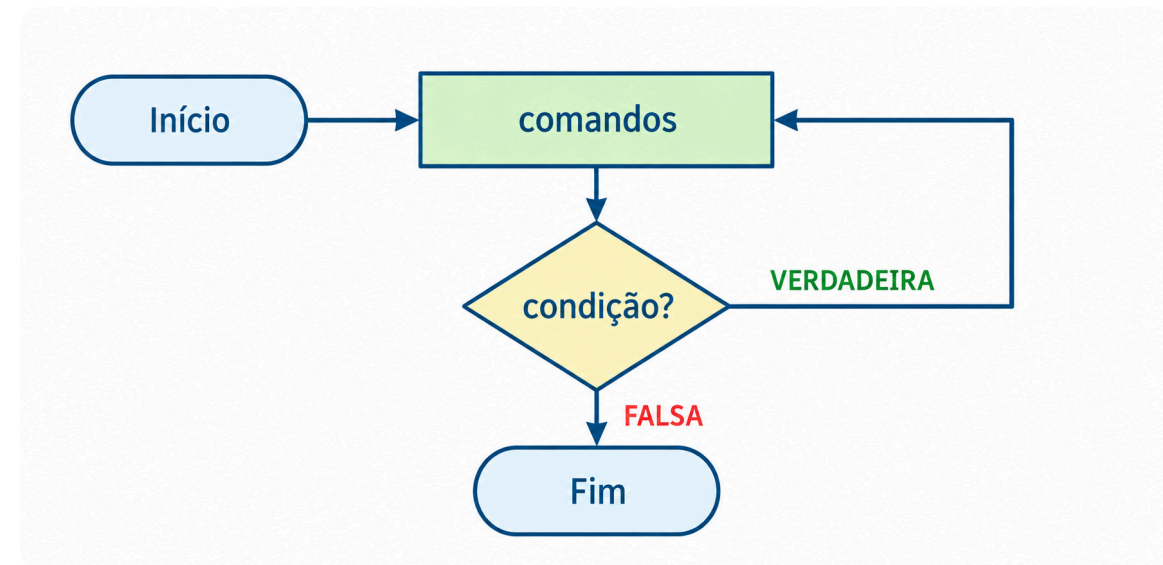
do...while

- Em C, o `do...while` é uma estrutura de repetição que **executa o bloco de código pelo menos uma vez** e só depois verifica a condição.

- A sintaxe básica é:

```
do {  
    // código a ser executado  
    // pelo menos uma vez  
} while (condição);
```

- A principal diferença é quando a condição é testada.
 - No `while`, a condição é testada **antes** de executar o bloco de código.
 - No `do...while`, a condição é testada **após** executar o bloco de código.



Controle de fluxo - Laços de repetição

do...while

- Exemplo de do...while:

```
int numero;
do {
    printf("Digite um número (negativo para sair): ");
    scanf("%d", &numero);
    if (numero >= 0) {
        printf("Número lido: %d\n", numero);
    }
} while (numero >= 0);

printf("Programa encerrado.\n");
```

- veja que podemos usar o do...while quando queremos garantir que o usuário tenha a oportunidade de fornecer pelo menos uma entrada antes de verificar a condição.

Controle de fluxo - Laços de repetição

for

- Vamos voltar ao exemplo de `while` com contador:

```
int numeros[5] = {20, 13, 42, 5, 18};  
int contador = 0;  
while (contador < 5) {  
    printf("Número: %d\n", numeros[contador]); // exibe o número atual  
    contador++; // incrementa o contador  
}
```

- Os casos em que usamos um contador, para controlar o número de iterações em um laço de repetição, são tão comuns que existe uma estrutura específica para isso: o laço `for`.

Controle de fluxo - Laços de repetição

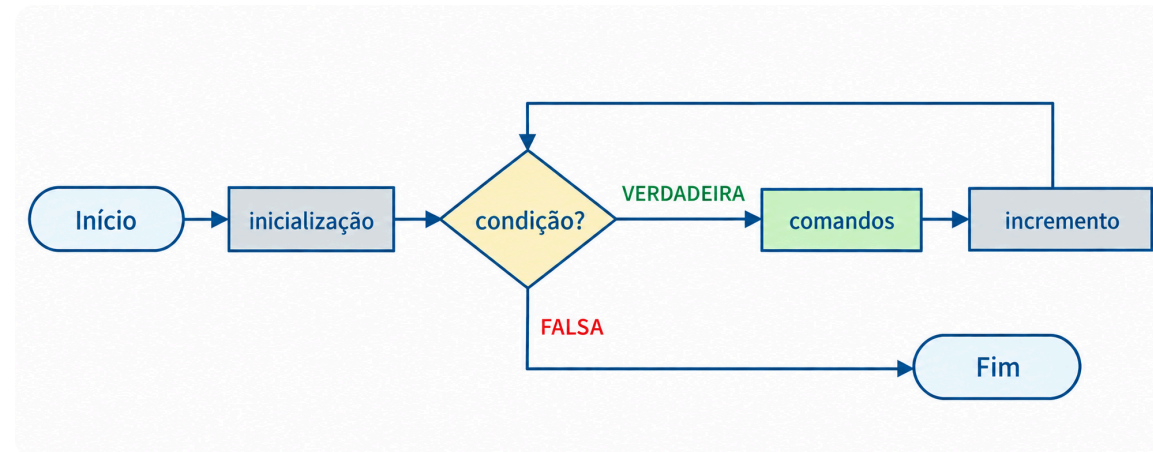
for

- A sintaxe do laço `for` é:

```
for (inicialização; condição; incremento) {  
    // código a ser executado  
}
```

- Exemplo com `for`:

```
int numeros[5] = {20, 13, 42, 5, 18};  
for (int i = 0; i < 5; i++) {  
    printf("Número: %d\n", numeros[i]);  
}
```



Operadores relacionais e lógicos

- Muitas vezes precisamos comparar valores e realizar operações que resultem em verdadeiro (true) ou falso (false) para, por exemplo, tomar decisões em nosso programa.
- Para isso, utilizamos **operadores relacionais** e **operadores lógicos**.
- **Operadores relacionais** comparam dois valores e retornam 1 ou 0:
 - `==` (igual)
 - `!=` (diferente)
 - `<` (menor)
 - `>` (maior)
 - `<=` (menor ou igual)
 - `>=` (maior ou igual)

Operadores relacionais e lógicos

- Não existe um tipo de dado booleano em C. Na verdade, o **valor 0**, em um contexto de expressão lógica, é considerado **falso**, e qualquer **valor diferente de 0** é considerado **verdadeiro**.
- Um operador relacional retorna 1, denotando verdadeiro, ou 0, denotando falso. Por exemplo:

```
int a = 5;
printf("%d\n", a == 5); // Saída: 1 (verdadeiro)
printf("%d\n", a != 5); // Saída: 0 (falso)
printf("%d\n", a < 10); // Saída: 1 (verdadeiro)
printf("%d\n", a > 10); // Saída: 0 (falso)
```

- Podemos usar **operadores relacionais em conjunto com operadores lógicos** para criar expressões mais complexas.

Operadores relacionais e lógicos

- **Operadores lógicos** combinam expressões booleanas e também retornam verdadeiro ou falso:
 - `&&` (E lógico): *retorna verdadeiro se **ambas** as expressões forem verdadeiras.*
 - `||` (OU lógico): *retorna verdadeiro se **pelo menos uma** das expressões for verdadeira.*
 - `!` (NÃO lógico): *inverte o valor lógico da expressão (verdadeiro se for falso, e falso se for verdadeiro).*
- Expressões conectadas por `&&` e `||` são avaliadas da esquerda para a direita, e a avaliação pode ser **curta-circuitada**:
 - Para `&&`, se a primeira expressão for falsa, a segunda não é avaliada.
 - Para `||`, se a primeira expressão for verdadeira, a segunda não é avaliada.
- **Exemplos:**

```
int a = 5, b = 10;
printf("%d\n", (a < 10) && (b > 5)); // Saída: 1 (verdadeiro)
printf("%d\n", (a > 10) || (b > 5)); // Saída: 1 (verdadeiro)
printf("%d\n", !(a == 5));           // Saída: 0 (falso)
```

Cadeia de caracteres

- Em C, não temos um tipo de dado específico para **strings**, como encontramos em outras linguagens.
- Em vez disso, usamos **vetores de caracteres** (arrays de `char`) para armazenar sequências de caracteres.
- Existem nuances importantes ao lidar com strings em C, como o fato de que elas são terminadas com um caractere nulo (`'\0'`), que indica o fim da string.
- Por exemplo, para declarar uma string que comporta até 10 caracteres, podemos fazer:

```
char nome[11];
```

- aqui, você deve estar se perguntando: *por que 11 e não 10?*
- isso ocorre porque precisamos *reservar espaço para o caractere nulo* (`'\0'`) que marca o fim da string.

Cadeia de caracteres

- Você pode inicializar uma string diretamente com uma sequência de caracteres entre aspas duplas:

```
char disciplina[] = "Estrutura de Dados";
```

- nesse caso, o compilador automaticamente adiciona o caractere nulo ao final da string.
- além disso, o tamanho do vetor é determinado pelo número de caracteres na string mais o caractere nulo.

- O exemplo acima é equivalente a:

```
char disciplina[19] = {'E', 's', 't', 'r', 'u', 't', 'u', 'r', 'a', ' ',  
                      'd', 'e', ' ', 'D', 'a', 'd', 'o', 's', '\\0'};
```

- aqui, estamos explicitamente definindo cada caractere da string, incluindo o caractere nulo no final.

Cadeia de caracteres

- Para ler uma string do usuário, podemos usar a função `scanf` com o especificador `%s`.

```
char nome[50];  
printf("Digite seu nome: ");  
scanf("%s", nome);  
printf("Seu nome é: %s\n", nome);
```

- um detalhe importante é que, ao usar `scanf` para ler strings, não precisamos usar o operador de endereço `&`, pois **o nome do vetor já representa o endereço do primeiro elemento**.
- No entanto, é importante saber que `scanf` lê apenas até o primeiro espaço em branco. Além disso, ele não respeita o tamanho do vetor, o que pode levar a **estouro de buffer**.
 - se você informar, por exemplo, "João da Silva", apenas "João" será lido e armazenado na variável `nome`.
 - se você digitar uma string maior que o tamanho do vetor, isso pode causar **comportamento indefinido**, pois o `scanf` não faz verificação de limites.

Cadeia de caracteres

- Para ler strings com espaços, respeitando o tamanho máximo do vetor, podemos usar a função `fgets`, que **lê uma linha inteira** até encontrar um caractere de nova linha (`\n`) ou atingir o tamanho máximo especificado.

```
char nome[50];  
printf("Digite seu nome completo: ");  
fgets(nome, sizeof(nome), stdin);  
printf("Seu nome completo é: %s", nome);
```

- nesse caso, `fgets` lê até 49 caracteres (o último espaço é reservado para o caractere nulo) e inclui o caractere de nova linha.

Cadeia de caracteres

Cuidados ao ler strings

- Algumas situações podem causar um comportamento inesperado ao ler strings, especialmente quando misturamos `scanf` e `fgets`.
- Mistura de `scanf` e `fgets`:
 - Teste esse código:

```
int idade;  
char nome[50];  
printf("Digite sua idade: ");  
scanf("%d", &idade);  
printf("Digite seu nome completo: ");  
fgets(nome, sizeof(nome), stdin);  
printf("Idade: %d, Nome: %s", idade, nome);
```

- nesse caso, o `fgets` vai capturar o `\n` deixado pelo `scanf` (o `[ENTER]` que foi pressionado), e a variável `nome` ficará com uma string vazia.

Cadeia de caracteres

Cuidados ao ler strings

- Mistura de `scanf` e `fgets` :

- Para evitar esse problema, podemos limpar o buffer de entrada antes de chamar `fgets` . Uma maneira de fazer isso é usando `getchar()` para consumir o caractere de nova linha deixado pelo `scanf` .

```
int idade;  
char nome[50];  
printf("Digite sua idade: ");  
scanf("%d", &idade);  
getchar(); // Limpa o buffer de entrada  
printf("Digite seu nome completo: ");  
fgets(nome, sizeof(nome), stdin);  
printf("Idade: %d, Nome: %s", idade, nome);
```

- Agora, o `getchar()` consome o caractere de nova linha, e o `fgets` vai capturar corretamente a entrada do usuário, e a variável `nome` terá o valor esperado.

Cadeia de caracteres

Cuidados ao ler strings

- `\n` no final da string lida com `fgets` :
 - Quando usamos `fgets`, o caractere de nova linha (`\n`) é incluído na string lida, caso haja espaço suficiente no buffer. Isso pode causar problemas se não for tratado corretamente.

```
char nome[50];  
printf("Digite seu nome completo: ");  
fgets(nome, sizeof(nome), stdin);  
printf("Nome lido: %s", nome);
```

```
char nome[50];  
printf("Digite seu nome completo: ");  
fgets(nome, sizeof(nome), stdin);  
nome[strcspn(nome, "\n")] = '\0';  
printf("Nome lido: %s", nome);
```

- *Você vai perceber que a string lida inclui o caractere de nova linha no final.*
- *Para remover o caractere de nova linha, podemos usar a função `strcspn` da biblioteca `string.h` para encontrar a posição do `\n` e substituí-lo por `\0`.*

Boas práticas em C

Comentários

- Comentários são trechos de código que não são executados.
- Podem ser utilizados para explicar o código ou para desativar temporariamente uma parte do código.
- **Comentário de uma linha:** `// comentário`
- **Comentário de múltiplas linhas:**

```
/* Comentário de múltiplas linhas
 * =====
 * Este tipo de comentário pode
 * se estender por várias linhas,
 * terminando com asterisco e barra.
 */
```

Boas práticas em C

Estrutura e Estilo de código

- Os programas abaixo irão compilar e executar normalmente, mas são difíceis de ler:

```
#include <stdio.h>
int main(void){int idade=20;printf("Idade: %d\n",idade);return 0;}
```

```
#include <stdio.h>

int      main(   void          ) {
    int  idade = 20;
        printf(
            "Idade: %d\n", idade
        );
return 0;
}
```

Boas práticas em C

Estrutura e Estilo de código

- Isso ocorre, porque em C praticamente todo espaço (espaço em branco, tabulação, quebra de linha) é ignorado pelo compilador.
- Mas isso não significa que devemos escrever código assim. Pelo contrário, devemos sempre buscar **clareza e legibilidade**. O estilo que você vai seguir pode variar, mas algumas boas práticas são:
 - Usar **indentação** para tornar mais clara a estrutura dos blocos de código (trecho delimitado por chaves).
 - Escrever **uma instrução por linha**.
 - Usar **nomes significativos** para variáveis e funções.
 - Evitar linhas muito longas, quebrando-as quando necessário.
 - Evitar **comentários óbvios**. Comente apenas o que for necessário para explicar a lógica do código.

Boas práticas em C

Estrutura e Estilo de código

- Um detalhe de estilo de código que geralmente divide opiniões é a posição das chaves `{}`. Existem duas abordagens comuns:

- **Estilo K&R (Kernighan & Ritchie):** a chave de abertura fica na mesma linha da declaração do bloco.

```
if (condição) {  
    // código  
} else {  
    // código  
}
```

- **Estilo Allman:** a chave de abertura fica na linha seguinte à declaração do bloco.

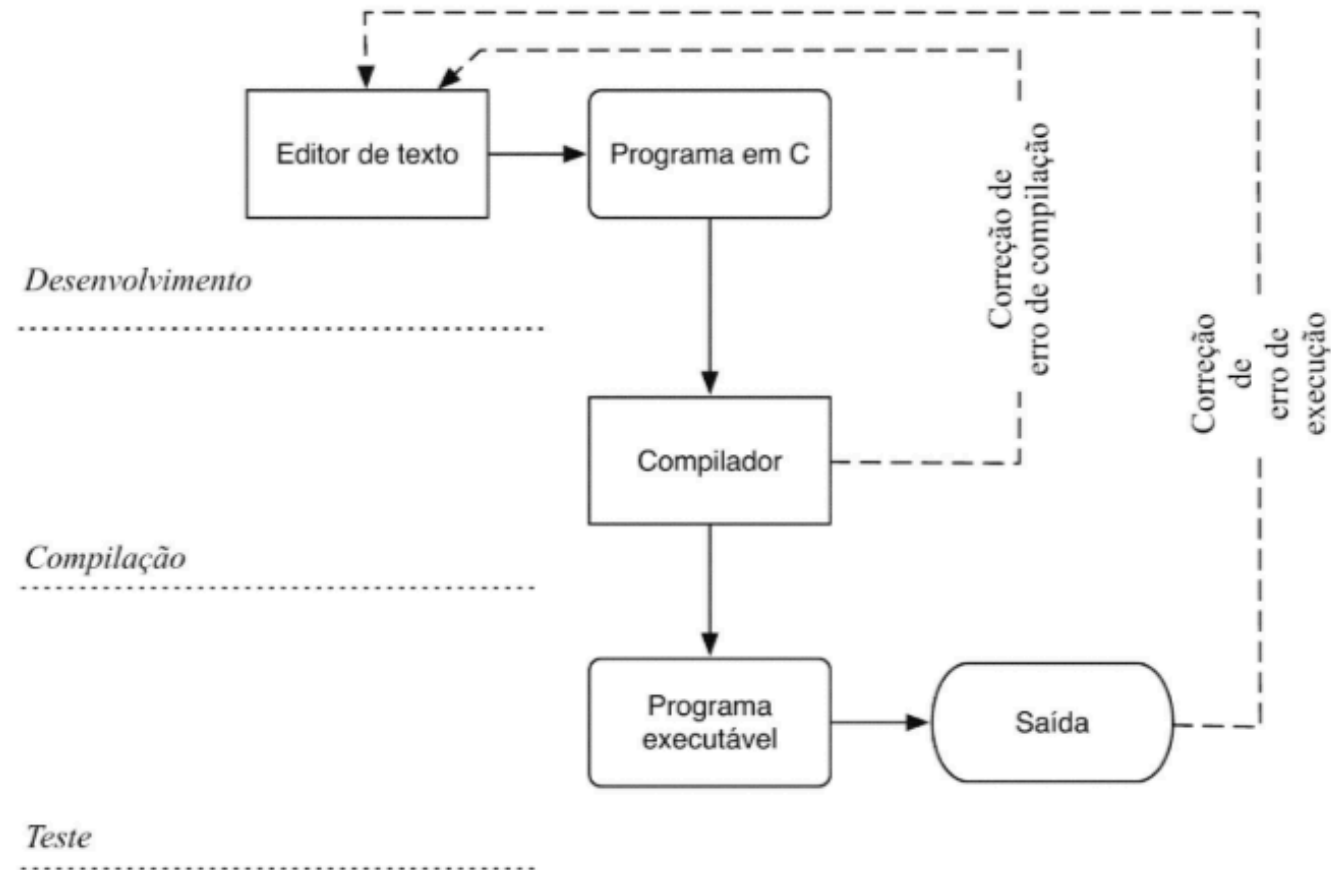
```
if (condição)  
{  
    // código  
} else  
{  
    // código  
}
```

Boas práticas em C

Leitura de strings

- Ao ler strings do usuário, é importante **respeitar o tamanho do vetor** para evitar estouro de buffer.
- Podemos usar `fgets` para ler strings com segurança, especificando o tamanho máximo do buffer.
- Ao misturar `scanf` e `fgets`, devemos limpar o buffer de entrada para evitar que o `fgets` capture o caractere de nova linha deixado pelo `scanf`.
- Ao usar `fgets`, devemos tratar o caractere de nova linha (`\n`) que é incluído na string lida, substituindo-o por um caractere nulo (`\0`) se necessário.

Ciclo de Desenvolvimento com C



Primeiros programas

- Utilizando uma ferramenta online, como o <https://www.programiz.com>, escreva o código abaixo e execute-o.

```
#include <stdio.h>

int main(void) {
    printf("Olá, mundo!\n");

    return 0;
}
```

- Agora, tente testar todos os exemplos de código apresentados nesta aula, alterando os valores e observando o resultado.

Preparar o ambiente de desenvolvimento

- Para programar em C, você precisa de um **compilador** e um **editor de código**.
- **Compilador:** é o programa que traduz o código-fonte em linguagem de máquina, gerando um arquivo executável. Alguns compiladores populares são:
 - ***GCC (GNU Compiler Collection)***
 - Clang
 - Microsoft Visual C++ (MSVC)
- Para nossa disciplina, vamos utilizar o **GCC**, que é gratuito e funciona em diferentes sistemas operacionais.
- **Editor de código:** é o programa que você usa para escrever o código-fonte. Na disciplina, vamos utilizar o **Visual Studio Code (VS Code)**, que é gratuito e amplamente utilizado na indústria.

Preparar o ambiente de desenvolvimento

Instalação do GCC no Windows

- Em sistemas Unix (Linux, macOS), o GCC geralmente já vem instalado.
- No Windows, você pode instalar o GCC através de pacotes ou ambientes de desenvolvimento, como o **MSYS2** (<https://www.msys2.org/>).
- O **MSYS2** fornece um ambiente Unix-like no Windows, incluindo o GCC e outras ferramentas de desenvolvimento.
- Siga as instruções de instalação no site oficial do MSYS2 para configurar o ambiente corretamente.

Se atente às instruções de instalação do MSYS2, pois é necessário instalar o GCC por meio do gerenciador de pacotes indicado no tutorial de instalação.

Preparar o ambiente de desenvolvimento

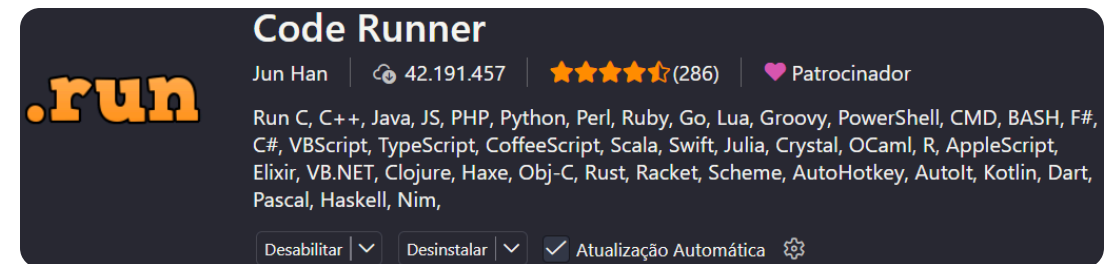
Instalação do GCC no Windows

- Por fim, adicione o caminho do executável do GCC à variável de ambiente `PATH` do Windows, para que você possa executar o compilador a partir de qualquer diretório no terminal.
- Geralmente, o executável do GCC estará localizado em um diretório como `C:\msys64\ucrt64\bin`. Adicione esse caminho à variável `PATH` nas configurações do sistema.
- Abra o terminal (Prompt de Comando ou PowerShell) e digite `gcc --version` para verificar se o GCC foi instalado corretamente. Você deve ver a versão do GCC instalada.

Preparar o ambiente de desenvolvimento

Configuração do Visual Studio Code (VS Code)

- Após instalar o VS Code, você pode instalar algumas extensões que facilitam o desenvolvimento em C, como:
 - **C/C++** (Microsoft): fornece suporte a depuração, IntelliSense e outras funcionalidades.
 - **Code Runner**: permite executar rapidamente trechos de código.



Preparar o ambiente de desenvolvimento

Teste do ambiente de desenvolvimento

- Para testar se o ambiente de desenvolvimento está funcionando corretamente, abra o VS Code, crie um novo arquivo com a extensão `.c`, e escreva um código simples, como o clássico "Olá, mundo!":
- Salve o arquivo e execute-o usando o Code Runner (irá aparecer um botão para executar, no canto superior direito); ou
- Você pode compilar manualmente no terminal com o comando `gcc nome_do_arquivo.c -o nome_do_executavel` e depois executar o programa gerado com `./nome_do_executavel.exe` (para Windows) ou `./nome_do_executavel` (para Linux/Mac).

O que aprendemos até aqui?

- Estrutura básica de um programa C
- Código-fonte e compilação
- Variáveis e tipos
- `printf` e `scanf`
- Operadores aritméticos, relacionais e lógicos
- Estruturas de controle de fluxo
- Relação entre tipo e memória
- `sizeof`
- Endereço de memória e operador `&`

Referências utilizadas

- **CELES, Waldemar; CERQUEIRA, Renato; RANGEL, José Lucas.** Introdução a Estruturas de Dados com Técnicas de Programação em C. 2ed. Rio de Janeiro: Elsevier, 2016.
- **DOBAY, Eduardo S.** *Programação em C*. São Paulo: USP, 2012.
- **PEREIRA, Silvio do Lago.** *Apostila de Linguagem C*. Publicação Independente.
- **MOREIRA NETO, Maurício.** *Linguagem de Programação — Linguagem C*. Slides sobre código-fonte, compilação, estrutura básica e primeiro programa.
- **MOREIRA NETO, Maurício.** *Tipos de Dados*. Slides sobre variáveis, tipos primitivos, `char`, `int`, `float` e `double`.